



Introdução Organização de código

Wagner Johnatan

Independente da linguagem, o programador começa criando uma **pasta raiz do projeto**. Dentro dela, organiza código, testes, documentação e configurações.



Estrutura padrão de um programa

```
meu-projeto/  
├──  
├── src/  
│   └── ...  
├── tests/  
├──  
├── tests automatizados  
├──  
├── docs/  
├──  
├── assets/  
├──  
├── config/  
├──  
├── data/  
├──  
├── .gitignore  
├── README.md  
└── arquivo de dependências/configuração da linguagem
```

Código-fonte da aplicação

Testes automatizados

Documentação do projeto

Imagens, ícones, fontes, arquivos estáticos

Configurações da aplicação

Dados locais, CSV, JSON etc. (quando necessário)

Arquivos que o Git deve ignorar

Explicação do projeto

arquivo de dependências/configuração da linguagem

O último arquivo varia conforme a tecnologia.

Python	→ requirements.txt / pyproject.toml
Node.js	→ package.json
Java Maven	→ pom.xml
Java Gradle	→ build.gradle
.NET / C#	→ .csproj
PHP Composer	→ composer.json
Rust	→ Cargo.toml
Go	→ go.mod

Criar a pasta raiz do projeto. Ela representa sua aplicação inteira. Exemplo: `sistema-clientes/`.

Abrir essa pasta na IDE. No VS Code, por exemplo, você abre `sistema-clientes`. A IDE passa a considerar essa pasta como seu workspace/projeto.

Inicializar a tecnologia escolhida. Algumas linguagens criam automaticamente parte da estrutura. Por exemplo:

Bash

npm init

Ou

Bash

dotnet new console

ou você pode simplesmente começar criando arquivos Python.

Criar ou utilizar a pasta `src/`. É aqui que deve ficar o código que você escreve.

`src/`
`main.py`

ou:

`src/`
`index.js`



Definir o ponto de entrada da aplicação. Praticamente todo programa precisa ter algum arquivo inicial. Dependendo da linguagem, pode ser:

main.py

main.java

Program.cs

index.js

main.go



Separar o código conforme o projeto cresce. Em vez de colocar tudo no **main**, você passa a criar módulos, classes ou componentes.

src/

main.py

usuarios/

produtos/

servicos/

Adicionar dependências externas quando necessário. Se sua aplicação precisa acessar banco de dados, API, interface gráfica etc., você instala bibliotecas e registra essas dependências no arquivo apropriado da linguagem.

Criar **tests/** para os testes. Idealmente, os testes ficam separados do código principal.

tests/

test_usuario.py

test_produto.py



Criar arquivos de configuração e variáveis de ambiente quando necessário. Por exemplo:

`.env`

Pode conter:

DATABASE_URL=...
API_KEY=...

Senhas e chaves **não devem ser colocadas diretamente no código.**



Configurar o Git. Normalmente:

git init

E criar:

.gitignore

para impedir que senhas, dependências instaladas e arquivos temporários sejam enviados ao repositório.

11. **Criar um README.md.** Ele deve responder rapidamente: o que é o projeto, como instalar, como executar e quais tecnologias foram utilizadas.

Não existe uma estrutura universal obrigatória.

Simples

```
calculadora/  
|  
├── src/  
|   └── main.py  
|  
├── README.md  
└── .gitignore
```

Profissional

```
sistema/  
|  
├── src/  
|   ├── controllers/  
|   ├── services/  
|   ├── models/  
|   ├── repositories/  
|   └── main  
|  
├── tests/  
├── docs/  
├── config/  
├── assets/  
├── .env  
├── .gitignore  
├── README.md  
└── dependências
```

DESENVOLVIMENTO

Segurança no Front

Limite de Caracteres	Previne ataques de Buffer Overflow (no nível de aplicação) e envios desnecessários de grandes volumes de dados.
Campos Obrigatórios	Garante que o estado da aplicação seja consistente antes do processamento.
Máscaras de Entrada (Ex: CPF, Telefone)	Garante a padronização e evita a inserção de caracteres especiais onde não deveriam existir.
Tipagem de Inputs	Usar <code><input type="email"></code> ou <code>type="number"</code> ajuda o navegador a validar o formato antes mesmo do clique no "Enviar".

A Tríade CID no Back-End

Confidencialidade: Garantir que o código do Back-End não vazze chaves de API ou segredos do sistema em respostas de erro.

Integridade: Validar os cálculos no servidor (Ex: nunca confiar no preço total enviado pelo Front-End; o Back-End deve calcular o valor real com base no banco de dados).

Disponibilidade: Implementar balanceamento de carga para que o processamento pesado não derrube o servidor.



AVALIAÇÃO DO 1º CICLO

1. Crie ou reutilize uma aplicação criada em sala;
2. Aplique o checklist das funções obrigatórias;
3. Crie a estrutura base em vibe coding mas traga para uma IDE;

Obs. Todas as modificações precisam ficar registradas em commit e devidamente versionadas, é individual e vale 1,0 ponto. Data de entrega **18/09** dia da avaliação. Deve ser apresentado individualmente.



Refatorando com IA

Escolha sua Extensão de IA

Se você ainda não tem uma, aqui estão as opções mais populares:

- **Codeium ou Tabnine:** Excelentes opções com planos gratuitos generosos.
- **GitHub Copilot:** A mais famosa e integrada, mas exige assinatura paga (gratuita para estudantes).
- **Gemini Code Assist:** A ferramenta do Google voltada para desenvolvedores.



2. Como Refatorar na Prática (O Passo a Passo)

A maioria dessas extensões funciona de duas formas principais: **Chat Lateral** ou **Chat Inline (no meio do código)**. O método *Inline* é o mais rápido para refatorar.

Passo 1: Selecione o código Use o mouse para selecionar exatamente a função, classe ou bloco de código que está confuso e você deseja melhorar.

Passo 2: Chame a IA

- **Método Inline (Recomendado):** Aperte **Ctrl + I** (Windows/Linux) ou **Cmd + I** (Mac). Uma pequena caixa de texto vai aparecer flutuando logo acima do seu código.
- **Método Chat Sidebar:** Clique no ícone da extensão na barra lateral esquerda e referencie o código selecionado.



Passo 3: Escreva o Prompt (Comando) Não peça apenas "melhore isso". Seja específico sobre o que você quer alcançar.

Aqui estão exemplos de **excelentes comandos de refatoração**:

- *"Refatore esta função para reduzir a complexidade (remova ifs aninhados usando early returns)."*
- *"Reescreva este trecho usando async/await em vez de .then() e .catch()."*
- *"Extraia as regras de negócio desta função gigante em funções menores e mais descritivas."*
- *"Refatore este código para deixá-lo mais legível e adicione comentários explicando a lógica."*
- *"Renomeie as variáveis deste código para que os nomes fiquem mais claros em português."*

Passo 4: Revise e Aceite (Accept/Discard) A IA vai gerar um "Diff" (uma visão de antes e depois), mostrando o que ela quer apagar em vermelho e o que quer adicionar em verde. **Leia com atenção.** Se estiver bom, clique em *Accept* (Aceitar).

Regras de Ouro ao Refatorar com IA

1. **A IA pode quebrar seu código:** Lembre-se da regra de ouro: refatorar **não pode** mudar o comportamento do programa. Às vezes, a IA se empolga e muda a lógica sem querer.
2. **Tenha testes ou faça testes manuais:** Antes de aceitar a refatoração, garanta que você consegue rodar o código para ver se ele continua entregando o mesmo resultado.
3. **Use o Git a seu favor:** Faça um *commit* do seu código funcionando **antes** de começar a brincar com a IA. Se a refatoração der errado, você simplesmente desfaz as mudanças.

O que estudamos no 1º Ciclo?

Processos de qualidade & Chat com IA



AValiação DO 1º CICLO

1. Vocês devem baixar o repositório que será refatorado;
2. Escolha qual SQUAD você irá refatorar e remova o restante;
3. Identifique as melhorias necessárias para a aplicação rodar corretamente, e **REFATORE** usando IA;
4. Adicione um chat que use IA;

Obs. Todas as modificações precisam ficar registradas em commit e devidamente versionadas, é individual e vale 1,0 ponto. Data de entrega **17/09** dia da avaliação. Deve ser apresentado individualmente.



Obrigado!!

